DBMS MINI PROJECT – PHASE 1

Semantic E-Commerce Engine & Supply Chain Hub

Database Design Report — Final Submission

1. Abstract

The Semantic E-Commerce Engine & Supply Chain Hub is a database design for an integrated e-commerce and supply-chain environment. Phase 1 focuses on the conceptual ER model and its exact relational mapping. The model represents customers, products, shopping carts, orders, order items, payments, warehouses, inventory, shipments, deliveries, reviews, and the many-to-many browsing relationship. The design uses Chen notation, explicit weak entities, composite/partial keys, primary and foreign keys, and cardinality constraints. It is structured as a normalized foundation for later SQL implementation and future semantic-search capabilities.

2. Problem Statement

An e-commerce platform must maintain consistent information about customers, products, carts, orders, individual order items, payments, warehouse stock, shipments, deliveries, and reviews. If these data are modeled independently, duplication and referential inconsistencies can occur. The proposed database provides a structured conceptual and relational model that preserves dependencies between business objects and supports traceability from a customer's order through shipment and delivery.

3. Objectives

- Design a complete Chen-notation ER model for the e-commerce and supply-chain workflow.
- Represent customers, products, carts, orders, order items, payments, warehouses, inventory, shipments, deliveries, and reviews.
- Represent the CUSTOMER–PRODUCT browsing M:N relationship using the BROWSES relation during ER-to-relational conversion.
- Identify weak/dependent entities and use appropriate composite keys.
- Map all 1:N and 1:1 relationships through foreign keys and uniqueness constraints.
- Maintain referential integrity and avoid ambiguous SQL names by using CUSTOMER_ORDER instead of ORDER.
- Produce an exact, implementation-ready relational schema that matches the supplied ER diagram without adding or removing business concepts.

4. Scope

- Customer profile and account information.
- Product catalogue information.
- Shopping cart ownership and status.
- Customer order placement and order details.

- Individual order-product lines.
- Payment recording.
- Warehouse and product inventory management.
- Shipment generation and delivery tracking.
- Customer order and delivery review information.
- Product browsing relationships.
- Future semantic-search/vector retrieval is outside the mandatory Phase 1 implementation.

5. ER Model — Chen Notation

The supplied ER diagram uses Chen notation. Rectangles represent strong entities, double rectangles represent weak entities, diamonds represent relationships, ovals represent attributes, underlined attributes represent primary keys, and dashed-underlined attributes represent partial keys. Cardinality labels are shown beside relationship lines.

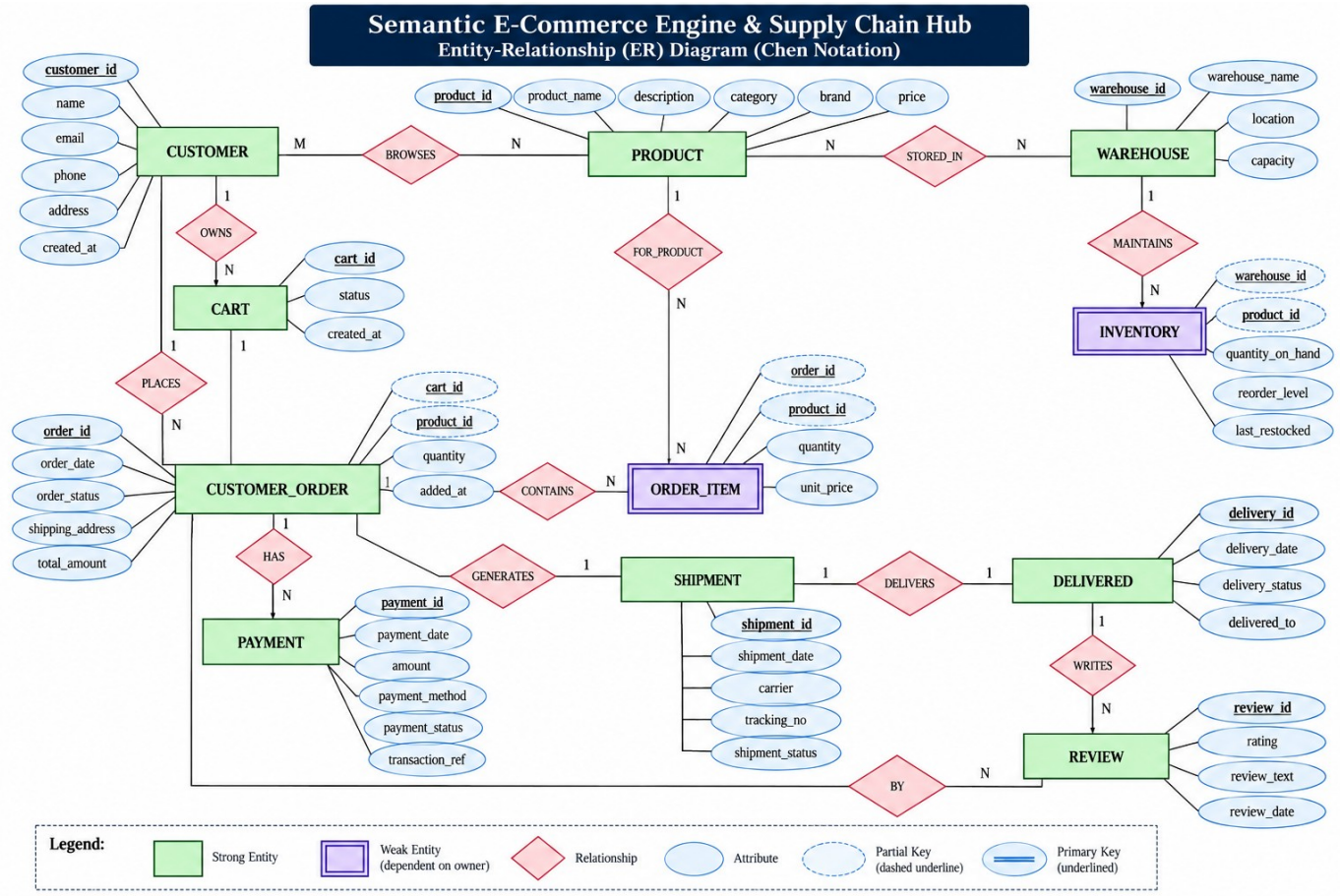


Figure 1. Final Chen-notation ER diagram used as the source model for the relational schema.

5.1 Strong Entities

Entity	Purpose	Primary Key
CUSTOMER	Stores customer identity and contact information.	customer_id
PRODUCT	Stores product catalogue information.	product_id
WAREHOUSE	Stores warehouse information.	warehouse_id
CART	Stores a customer's shopping cart.	cart_id
CUSTOMER_ORDER	Stores a customer's placed order.	order_id
PAYMENT	Stores payment information for an order.	payment_id
SHIPMENT	Stores shipment and tracking information.	shipment_id
DELIVERED	Stores delivery completion information for a shipment.	delivery_id
REVIEW	Stores review information associated with an order and its delivery.	review_id

5.2 Weak / Dependent Entities

Entity	Owner / Identifying Context	Composite Primary Key	Reason
ORDER_ITEM	CUSTOMER_ORDER	(order_id, product_id)	An order item is dependent on its parent order and identifies a product within that order.
INVENTORY	WAREHOUSE + PRODUCT	(warehouse_id, product_id)	A stock record is identified by the warehouse-product combination.

BROWSES is an M:N relationship in the ER model, not an additional strong/weak entity. During relational conversion, it becomes the BROWSES relation containing the primary keys of CUSTOMER and PRODUCT.

5.3 Relationships and Cardinalities

Relationship	Cardinality	Relational Mapping
CUSTOMER — OWNS — CART	1 : N	CART.customer_id → CUSTOMER.customer_id
CUSTOMER — BROWSES — PRODUCT	M : N	BROWSES(customer_id, product_id)
PRODUCT — STORED_IN — WAREHOUSE	N : N	INVENTORY(warehouse_id, product_id)
WAREHOUSE — MAINTAINS —	1 : N	INVENTORY.warehouse_id →

INVENTORY		WAREHOUSE.warehouse_id
PRODUCT — FOR_PRODUCT — ORDER_ITEM	1 : N	ORDER_ITEM.product_id → PRODUCT.product_id
CUSTOMER — PLACES — CUSTOMER_ORDER	1 : N	CUSTOMER_ORDER.customer_id → CUSTOMER.customer_id
CUSTOMER_ORDER — CONTAINS — ORDER_ITEM	1 : N	ORDER_ITEM.order_id → CUSTOMER_ORDER.order_id
CUSTOMER_ORDER — HAS — PAYMENT	1 : N	PAYMENT.order_id → CUSTOMER_ORDER.order_id
CUSTOMER_ORDER — GENERATES — SHIPMENT	1 : 1	SHIPMENT.order_id → CUSTOMER_ORDER.order_id; UNIQUE
SHIPMENT — DELIVERS — DELIVERED	1 : 1	DELIVERED.shipment_id → SHIPMENT.shipment_id; UNIQUE
CUSTOMER_ORDER — BY — REVIEW	1 : N	REVIEW.order_id → CUSTOMER_ORDER.order_id
DELIVERED — WRITES — REVIEW	1 : N	REVIEW.delivery_id → DELIVERED.delivery_id

6. Entity Attributes

Relation	**Attributes**
CUSTOMER	customer_id (PK), name, email (UNIQUE), phone, address, created_at
PRODUCT	product_id (PK), product_name, description, category, brand, price
WAREHOUSE	warehouse_id (PK), warehouse_name, location, capacity
CART	cart_id (PK), status, created_at; customer_id is added as FK from OWNS
INVENTORY	warehouse_id (PK/FK), product_id (PK/FK), quantity_on_hand, reorder_level, last_restocked
CUSTOMER_ORDER	order_id (PK), order_date, order_status, shipping_address, total_amount; customer_id is added as FK from PLACES
ORDER_ITEM	order_id (PK/FK), product_id (PK/FK), quantity, unit_price
PAYMENT	payment_id (PK), payment_date, amount, payment_method, payment_status, transaction_ref; order_id is added as FK from HAS
SHIPMENT	shipment_id (PK), shipment_date, carrier, tracking_no, shipment_status; order_id is added as UNIQUE FK from GENERATES
DELIVERED	delivery_id (PK), delivery_date, delivery_status, delivered_to; shipment_id is added as UNIQUE FK from DELIVERS
REVIEW	review_id (PK), rating, review_text, review_date; order_id and delivery_id are added as FKs from BY and WRITES

7. Final Relational Schema

The following schema is derived directly from the supplied ER diagram. Foreign keys introduced by relationship mapping are explicitly shown. No CART_ITEM relation, no WAREHOUSE–SHIPMENT relation, no dispatch_date attribute, and no direct PRODUCT–REVIEW relationship are included because they are not present in the supplied ER diagram.

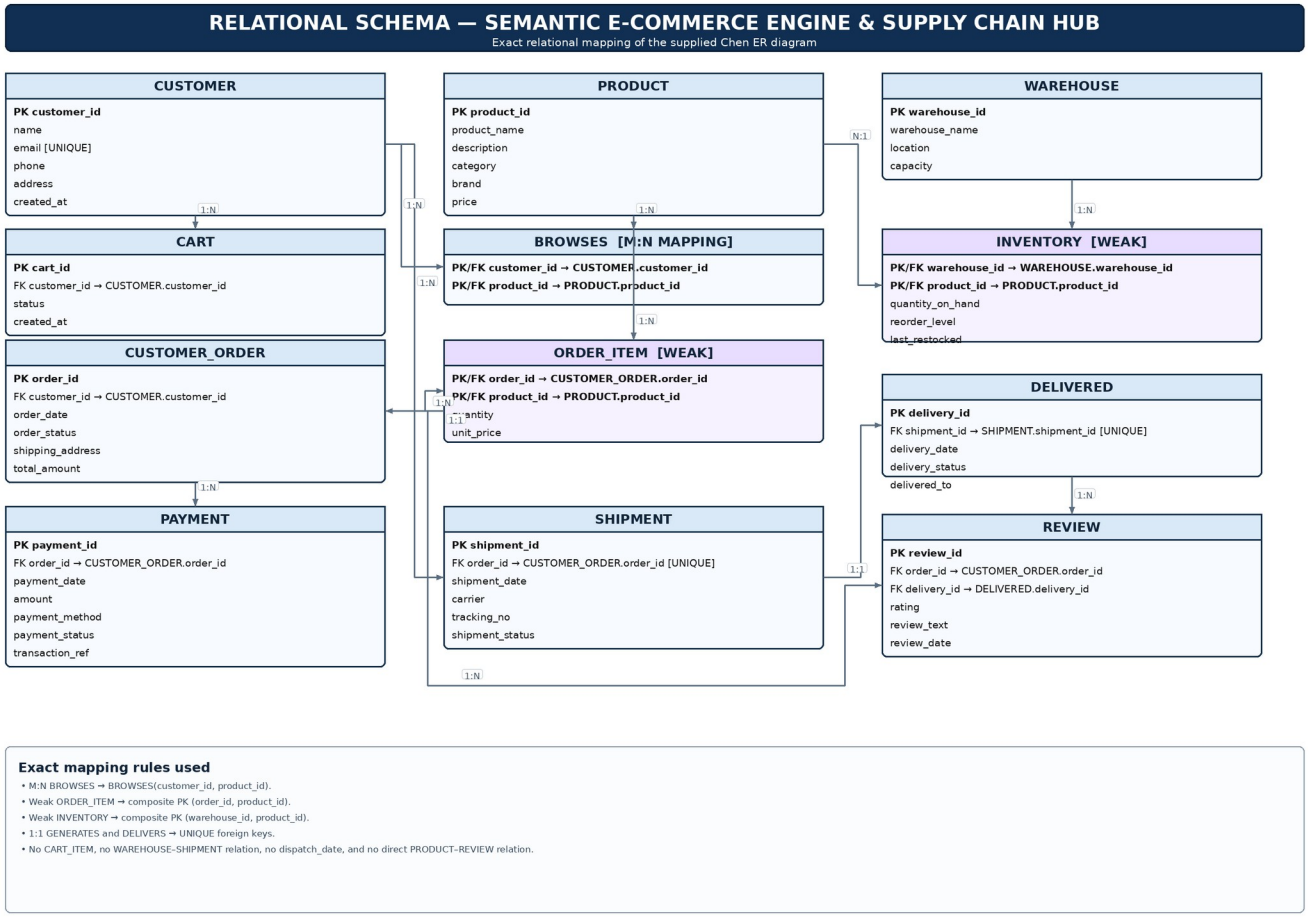


Figure 2. Final relational schema — exact mapping of the supplied ER diagram.

7.1 Relation Definitions

Relation	Final Definition
CUSTOMER	customer_id PK; name; email UNIQUE; phone; address; created_at
PRODUCT	product_id PK; product_name; description; category; brand; price
WAREHOUSE	warehouse_id PK; warehouse_name; location; capacity
CART	cart_id PK; customer_id FK → CUSTOMER.customer_id; status; created_at
BROWSES	customer_id PK/FK → CUSTOMER.customer_id; product_id PK/FK → PRODUCT.product_id
INVENTORY	warehouse_id PK/FK → WAREHOUSE.warehouse_id; product_id PK/FK → PRODUCT.product_id; quantity_on_hand;

	reorder_level; last_restocked
CUSTOMER_ORDER	order_id PK; customer_id FK → CUSTOMER.customer_id; order_date; order_status; shipping_address; total_amount
ORDER_ITEM	order_id PK/FK → CUSTOMER_ORDER.order_id; product_id PK/FK → PRODUCT.product_id; quantity; unit_price
PAYMENT	payment_id PK; order_id FK → CUSTOMER_ORDER.order_id; payment_date; amount; payment_method; payment_status; transaction_ref
SHIPMENT	shipment_id PK; order_id UNIQUE FK → CUSTOMER_ORDER.order_id; shipment_date; carrier; tracking_no; shipment_status
DELIVERED	delivery_id PK; shipment_id UNIQUE FK → SHIPMENT.shipment_id; delivery_date; delivery_status; delivered_to
REVIEW	review_id PK; order_id FK → CUSTOMER_ORDER.order_id; delivery_id FK → DELIVERED.delivery_id; rating; review_text; review_date

8. Why BROWSES Appears in the Relational Schema

BROWSES is drawn as a relationship diamond in the ER diagram, not as a separate entity. Because CUSTOMER and PRODUCT participate in a many-to-many relationship, the standard ER-to-relational transformation creates a separate relation named BROWSES. Its composite primary key is (customer_id, product_id), and both columns are also foreign keys. Therefore, BROWSES is not an extra business entity; it is the relational representation of the M:N relationship shown in the ER diagram.

9. Why CUSTOMER_ORDER and ORDER_ITEM Are Different

CUSTOMER_ORDER represents the order-level transaction: the customer who placed it, order date, status, shipping address, and total amount. ORDER_ITEM represents an individual product line within that order: the product, quantity, and unit price. Therefore, one CUSTOMER_ORDER can contain multiple ORDER_ITEM records. ORDER_ITEM is dependent on CUSTOMER_ORDER and uses the composite key (order_id, product_id).

10. Weak Entity and Composite-Key Justification

ORDER_ITEM depends on CUSTOMER_ORDER for its identity, so order_id participates in its primary key. INVENTORY depends on the warehouse-product combination, so (warehouse_id, product_id) forms its composite primary key. These keys prevent duplicate product lines within the same order and duplicate stock records for the same product in the same warehouse.

11. Functional Flow

- A CUSTOMER owns one or more CART records.
- A CUSTOMER browses many PRODUCT records, and a PRODUCT can be browsed by many customers; this is represented by BROWSES.

- PRODUCTs are stored in WAREHOUSEs; INVENTORY records identify each warehouse-product stock combination.
- A CUSTOMER places a CUSTOMER_ORDER.
- A CUSTOMER_ORDER contains ORDER_ITEM records, each identifying a PRODUCT and its quantity and unit price.
- A CUSTOMER_ORDER can have PAYMENT records.
- A CUSTOMER_ORDER generates one SHIPMENT in the Phase 1 ER model.
- A SHIPMENT delivers one DELIVERED record.
- A REVIEW is associated with a CUSTOMER_ORDER through BY and with a DELIVERED record through WRITES.

12. SQL Database Implementation — DDL and DML

This section contains the SQL implementation corresponding to the final ER diagram and relational schema. The script creates the database, defines all relations and constraints, inserts sample data, and provides verification queries. The SQL is intentionally kept consistent with the Phase 1 design; no additional business entity or relationship is introduced.

12.1 DDL Script

The following DDL statements create the database and all 12 relations: CUSTOMER, PRODUCT, WAREHOUSE, CART, BROWSES, INVENTORY, CUSTOMER_ORDER, ORDER_ITEM, PAYMENT, SHIPMENT, DELIVERED, and REVIEW. Primary keys, foreign keys, composite keys, UNIQUE constraints, NOT NULL constraints, and CHECK constraints are included.

```
DROP DATABASE IF EXISTS semantic_ecommerce_hub;
CREATE DATABASE semantic_ecommerce_hub;
USE semantic_ecommerce_hub;

-- ============================================================
-- 1. CUSTOMER
-- ============================================================
CREATE TABLE CUSTOMER (
    customer_id INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) NOT NULL UNIQUE,
    phone VARCHAR(20),
    address VARCHAR(255),
    created_at DATETIME NOT NULL
);

-- ============================================================
-- 2. PRODUCT
-- ============================================================
CREATE TABLE PRODUCT (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(150) NOT NULL,
    description VARCHAR(500),
    category VARCHAR(100),
    brand VARCHAR(100),
    price DECIMAL(10,2) NOT NULL,
    CONSTRAINT chk_product_price CHECK (price >= 0)
);

-- ============================================================
-- 3. WAREHOUSE
-- ============================================================
CREATE TABLE WAREHOUSE (
    warehouse_id INT PRIMARY KEY,
    warehouse_name VARCHAR(100) NOT NULL,
    location VARCHAR(255),
    capacity INT,
    CONSTRAINT chk_warehouse_capacity CHECK (capacity >= 0)
```

```
);

-- ============================================================
-- 4. CART
-- CUSTOMER 1:N CART
-- ============================================================
CREATE TABLE CART (
    cart_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    status VARCHAR(30) NOT NULL,
    created_at DATETIME NOT NULL,
    CONSTRAINT fk_cart_customer
        FOREIGN KEY (customer_id)
        REFERENCES CUSTOMER(customer_id)
);

-- ============================================================
-- 5. BROWSES
-- M:N CUSTOMER-PRODUCT relationship mapping
-- Composite PK = (customer_id, product_id)
-- ============================================================
CREATE TABLE BROWSES (
    customer_id INT NOT NULL,
    product_id INT NOT NULL,
    PRIMARY KEY (customer_id, product_id),
    CONSTRAINT fk_browses_customer
        FOREIGN KEY (customer_id)
        REFERENCES CUSTOMER(customer_id),
    CONSTRAINT fk_browses_product
        FOREIGN KEY (product_id)
        REFERENCES PRODUCT(product_id)
);

-- ============================================================
-- 6. INVENTORY
-- Weak / dependent relation
-- Composite PK = (warehouse_id, product_id)
-- ============================================================
CREATE TABLE INVENTORY (
    warehouse_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity_on_hand INT NOT NULL,
    reorder_level INT NOT NULL,
    last_restocked DATE,
    PRIMARY KEY (warehouse_id, product_id),
    CONSTRAINT fk_inventory_warehouse
        FOREIGN KEY (warehouse_id)
        REFERENCES WAREHOUSE(warehouse_id),
    CONSTRAINT fk_inventory_product
        FOREIGN KEY (product_id)
        REFERENCES PRODUCT(product_id),
    CONSTRAINT chk_inventory_quantity CHECK (quantity_on_hand >= 0),
    CONSTRAINT chk_inventory_reorder CHECK (reorder_level >= 0)
);

-- ============================================================
-- 7. CUSTOMER_ORDER
-- CUSTOMER 1:N CUSTOMER_ORDER
-- ============================================================
CREATE TABLE CUSTOMER_ORDER (
    order_id INT PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date DATETIME NOT NULL,
    order_status VARCHAR(30) NOT NULL,
    shipping_address VARCHAR(255) NOT NULL,
    total_amount DECIMAL(12,2) NOT NULL,
    CONSTRAINT fk_order_customer
        FOREIGN KEY (customer_id)
        REFERENCES CUSTOMER(customer_id),
    CONSTRAINT chk_order_total CHECK (total_amount >= 0)
);
```

```
-- ============================================================
-- 8. ORDER_ITEM
-- Weak / dependent relation
-- Composite PK = (order_id, product_id)
-- ============================================================
CREATE TABLE ORDER_ITEM (
    order_id INT NOT NULL,
    product_id INT NOT NULL,
    quantity INT NOT NULL,
    unit_price DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (order_id, product_id),
    CONSTRAINT fk_order_item_order
        FOREIGN KEY (order_id)
        REFERENCES CUSTOMER_ORDER(order_id),
    CONSTRAINT fk_order_item_product
        FOREIGN KEY (product_id)
        REFERENCES PRODUCT(product_id),
    CONSTRAINT chk_order_item_quantity CHECK (quantity > 0),
    CONSTRAINT chk_order_item_price CHECK (unit_price >= 0)
);

-- ============================================================
-- 9. PAYMENT
-- CUSTOMER_ORDER 1:N PAYMENT
-- ============================================================
CREATE TABLE PAYMENT (
    payment_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    payment_date DATETIME NOT NULL,
    amount DECIMAL(12,2) NOT NULL,
    payment_method VARCHAR(40) NOT NULL,
    payment_status VARCHAR(30) NOT NULL,
    transaction_ref VARCHAR(100),
    CONSTRAINT fk_payment_order
        FOREIGN KEY (order_id)
        REFERENCES CUSTOMER_ORDER(order_id),
    CONSTRAINT chk_payment_amount CHECK (amount >= 0)
);

-- ============================================================
-- 10. SHIPMENT
-- CUSTOMER_ORDER 1:1 SHIPMENT
-- UNIQUE(order_id) enforces the 1:1 relationship
-- ============================================================
CREATE TABLE SHIPMENT (
    shipment_id INT PRIMARY KEY,
    order_id INT NOT NULL UNIQUE,
    shipment_date DATE,
    carrier VARCHAR(100),
    tracking_no VARCHAR(100),
    shipment_status VARCHAR(30) NOT NULL,
    CONSTRAINT fk_shipment_order
        FOREIGN KEY (order_id)
        REFERENCES CUSTOMER_ORDER(order_id)
);

-- ============================================================
-- 11. DELIVERED
-- SHIPMENT 1:1 DELIVERED
-- UNIQUE(shipment_id) enforces the 1:1 relationship
-- ============================================================
CREATE TABLE DELIVERED (
    delivery_id INT PRIMARY KEY,
    shipment_id INT NOT NULL UNIQUE,
    delivery_date DATE,
    delivery_status VARCHAR(30) NOT NULL,
    delivered_to VARCHAR(100),
    CONSTRAINT fk_delivered_shipment
        FOREIGN KEY (shipment_id)
        REFERENCES SHIPMENT(shipment_id)
```

```
);

-- ============================================================
-- 12. REVIEW
-- CUSTOMER_ORDER 1:N REVIEW through BY
-- DELIVERED 1:N REVIEW through WRITES
-- ============================================================
CREATE TABLE REVIEW (
    review_id INT PRIMARY KEY,
    order_id INT NOT NULL,
    delivery_id INT NOT NULL,
    rating INT NOT NULL,
    review_text VARCHAR(500),
    review_date DATE NOT NULL,
    CONSTRAINT fk_review_order
        FOREIGN KEY (order_id)
        REFERENCES CUSTOMER_ORDER(order_id),
    CONSTRAINT fk_review_delivery
        FOREIGN KEY (delivery_id)
        REFERENCES DELIVERED(delivery_id),
    CONSTRAINT chk_review_rating CHECK (rating BETWEEN 1 AND 5)
);
```

12.2 DML Script — Sample Data

The following INSERT statements populate every relation with representative data while respecting the foreign-key dependencies and composite-key constraints.

```
-- ============================================================
-- INSERT SAMPLE DATA
-- ============================================================

-- 1. CUSTOMER
INSERT INTO CUSTOMER
(customer_id, name, email, phone, address, created_at)
VALUES
(101, 'Rahul Sharma', 'rahul.sharma@example.com', '9876501001',
 'Bengaluru, Karnataka', '2026-09-01 09:30:00'),
(102, 'Ananya Rao', 'ananya.rao@example.com', '9876501002',
 'Mysuru, Karnataka', '2026-09-02 10:15:00'),
(103, 'Vikram Kumar', 'vikram.kumar@example.com', '9876501003',
 'Chennai, Tamil Nadu', '2026-09-03 11:00:00'),
(104, 'Sneha Patil', 'sneha.patil@example.com', '9876501004',
 'Pune, Maharashtra', '2026-09-04 14:20:00'),
(105, 'Arjun Mehta', 'arjun.mehta@example.com', '9876501005',
 'Hyderabad, Telangana', '2026-09-05 16:45:00');

-- 2. PRODUCT
INSERT INTO PRODUCT
(product_id, product_name, description, category, brand, price)
VALUES
(201, 'Lenovo IdeaPad Laptop', '15.6-inch laptop for students and professionals',
 'Laptop', 'Lenovo', 58999.00),
(202, 'Wireless Mechanical Keyboard', 'Compact mechanical keyboard with wireless connectivity',
 'Accessories', 'KeyPro', 4999.00),
(203, 'Noise Cancelling Headphones', 'Over-ear headphones with active noise cancellation',
 'Audio', 'SoundMax', 7999.00),
(204, 'USB-C Hub', 'Multi-port USB-C hub with HDMI and USB ports',
 'Accessories', 'TechLink', 2499.00),
(205, 'Smartphone', '5G smartphone with 128 GB storage',
 'Mobile', 'NovaTech', 24999.00),
(206, 'Wireless Mouse', 'Ergonomic wireless mouse',
 'Accessories', 'LogiTech', 1599.00);

-- 3. WAREHOUSE
INSERT INTO WAREHOUSE
(warehouse_id, warehouse_name, location, capacity)
VALUES
(301, 'Bengaluru Central Warehouse', 'Bengaluru, Karnataka', 10000),
(302, 'Chennai Distribution Hub', 'Chennai, Tamil Nadu', 8000),
```

```
(303, 'Hyderabad Fulfillment Center', 'Hyderabad, Telangana', 7500);

-- 4. CART
INSERT INTO CART
(cart_id, customer_id, status, created_at)
VALUES
(401, 101, 'ACTIVE', '2026-09-10 18:00:00'),
(402, 102, 'CHECKED_OUT', '2026-09-11 12:30:00'),
(403, 103, 'ACTIVE', '2026-09-12 09:15:00'),
(404, 104, 'ABANDONED', '2026-09-13 20:10:00'),
(405, 105, 'CHECKED_OUT', '2026-09-14 15:25:00');

-- 5. BROWSES
INSERT INTO BROWSES
(customer_id, product_id)
VALUES
(101, 201),
(101, 202),
(101, 203),
(102, 201),
(102, 205),
(103, 203),
(103, 204),
(104, 202),
(104, 206),
(105, 201),
(105, 205),
(105, 206);

-- 6. INVENTORY
INSERT INTO INVENTORY
(warehouse_id, product_id, quantity_on_hand, reorder_level, last_restocked)
VALUES
(301, 201, 45, 10, '2026-09-15'),
(301, 202, 120, 25, '2026-09-16'),
(301, 203, 65, 15, '2026-09-14'),
(301, 204, 90, 20, '2026-09-17'),
(302, 201, 30, 10, '2026-09-13'),
(302, 205, 55, 12, '2026-09-15'),
(302, 206, 150, 30, '2026-09-16'),
(303, 203, 40, 10, '2026-09-12'),
(303, 204, 70, 15, '2026-09-14'),
(303, 205, 35, 10, '2026-09-15');

-- 7. CUSTOMER_ORDER
INSERT INTO CUSTOMER_ORDER
(order_id, customer_id, order_date, order_status, shipping_address, total_amount)
VALUES
(501, 101, '2026-09-18 10:30:00', 'DELIVERED',
 'Bengaluru, Karnataka', 63998.00),
(502, 102, '2026-09-19 13:15:00', 'DELIVERED',
 'Mysuru, Karnataka', 24999.00),
(503, 103, '2026-09-20 16:40:00', 'SHIPPED',
 'Chennai, Tamil Nadu', 10498.00),
(504, 104, '2026-09-21 11:20:00', 'PROCESSING',
 'Pune, Maharashtra', 4999.00),
(505, 105, '2026-09-22 17:05:00', 'DELIVERED',
 'Hyderabad, Telangana', 26598.00);

-- 8. ORDER_ITEM
INSERT INTO ORDER_ITEM
(order_id, product_id, quantity, unit_price)
VALUES
(501, 201, 1, 58999.00),
(501, 202, 1, 4999.00),
(502, 205, 1, 24999.00),
(503, 203, 1, 7999.00),
(503, 204, 1, 2499.00),
(504, 202, 1, 4999.00),
(505, 205, 1, 24999.00),
(505, 206, 1, 1599.00);
```

```
-- 9. PAYMENT
INSERT INTO PAYMENT
(payment_id, order_id, payment_date, amount, payment_method, payment_status, transaction_ref)
VALUES
(601, 501, '2026-09-18 10:35:00', 63998.00, 'UPI', 'SUCCESS', 'TXN501UPI'),
(602, 502, '2026-09-19 13:20:00', 24999.00, 'CARD', 'SUCCESS', 'TXN502CARD'),
(603, 503, '2026-09-20 16:45:00', 10498.00, 'UPI', 'SUCCESS', 'TXN503UPI'),
(604, 504, '2026-09-21 11:25:00', 4999.00, 'CARD', 'SUCCESS', 'TXN504CARD'),
(605, 505, '2026-09-22 17:10:00', 26598.00, 'NET_BANKING', 'SUCCESS', 'TXN505NB');

-- 10. SHIPMENT
-- Each order has exactly one shipment because order_id is UNIQUE.
INSERT INTO SHIPMENT
(shipment_id, order_id, shipment_date, carrier, tracking_no, shipment_status)
VALUES
(701, 501, '2026-09-19', 'BlueDart', 'BD501001', 'DELIVERED'),
(702, 502, '2026-09-20', 'Delhivery', 'DL502002', 'DELIVERED'),
(703, 503, '2026-09-21', 'DTDC', 'DT503003', 'IN_TRANSIT'),
(704, 504, '2026-09-22', 'BlueDart', 'BD504004', 'PROCESSING'),
(705, 505, '2026-09-23', 'Delhivery', 'DL505005', 'DELIVERED');

-- 11. DELIVERED
-- One delivery record per shipment because shipment_id is UNIQUE.
INSERT INTO DELIVERED
(delivery_id, shipment_id, delivery_date, delivery_status, delivered_to)
VALUES
(801, 701, '2026-09-21', 'DELIVERED', 'Rahul Sharma'),
(802, 702, '2026-09-22', 'DELIVERED', 'Ananya Rao'),
(803, 705, '2026-09-25', 'DELIVERED', 'Arjun Mehta');

-- 12. REVIEW
-- Reviews reference the order and the corresponding delivered record.
INSERT INTO REVIEW
(review_id, order_id, delivery_id, rating, review_text, review_date)
VALUES
(901, 501, 801, 5, 'Excellent laptop and fast delivery.', '2026-09-22'),
(902, 502, 802, 4, 'Good smartphone and smooth delivery.', '2026-09-23'),
(903, 505, 803, 5, 'Great product quality and packaging.', '2026-09-26');
```

1.3 Verification / Retrieval Queries

The following queries verify that the created relations contain data and that the main relationships can be retrieved correctly using JOIN operations.

```
-- ============================================================
-- VERIFICATION QUERIES
-- ============================================================

SHOW TABLES;

SELECT * FROM CUSTOMER;
SELECT * FROM PRODUCT;
SELECT * FROM WAREHOUSE;
SELECT * FROM CART;
SELECT * FROM BROWSES;
SELECT * FROM INVENTORY;
SELECT * FROM CUSTOMER_ORDER;
SELECT * FROM ORDER_ITEM;
SELECT * FROM PAYMENT;
SELECT * FROM SHIPMENT;
SELECT * FROM DELIVERED;
SELECT * FROM REVIEW;

-- Relationship verification
SELECT
    c.name AS customer_name,
    co.order_id,
    co.order_status,
    co.total_amount
```

```
FROM CUSTOMER c
JOIN CUSTOMER_ORDER co
    ON c.customer_id = co.customer_id;

SELECT
    co.order_id,
    p.product_name,
    oi.quantity,
    oi.unit_price
FROM CUSTOMER_ORDER co
JOIN ORDER_ITEM oi
    ON co.order_id = oi.order_id
JOIN PRODUCT p
    ON oi.product_id = p.product_id;

SELECT
    w.warehouse_name,
    p.product_name,
    i.quantity_on_hand,
    i.reorder_level
FROM INVENTORY i
JOIN WAREHOUSE w
    ON i.warehouse_id = w.warehouse_id
JOIN PRODUCT p
    ON i.product_id = p.product_id;

SELECT
    co.order_id,
    s.shipment_id,
    d.delivery_id,
    d.delivery_status
FROM CUSTOMER_ORDER co
JOIN SHIPMENT s
    ON co.order_id = s.order_id
LEFT JOIN DELIVERED d
    ON s.shipment_id = d.shipment_id;

SELECT
    c.name AS customer_name,
    p.product_name
FROM BROWSES b
JOIN CUSTOMER c
    ON b.customer_id = c.customer_id
JOIN PRODUCT p
    ON b.product_id = p.product_id;

SELECT
    r.review_id,
    co.order_id,
    d.delivery_id,
    r.rating,
    r.review_text
FROM REVIEW r
JOIN CUSTOMER_ORDER co
    ON r.order_id = co.order_id
JOIN DELIVERED d
    ON r.delivery_id = d.delivery_id;

-- ============================================================
-- END OF SCRIPT
-- ============================================================
```

12. Conclusion

The final Phase 1 database design provides a clean and consistent conceptual-to-relational mapping for the Semantic E-Commerce Engine & Supply Chain Hub. The relational schema is derived directly from the supplied Chen ER diagram. The design preserves the ER cardinalities, represents the M:N BROWSES relationship as a relational mapping, uses composite keys for dependent records, and uses UNIQUE foreign

keys for the two 1:1 relationships. No additional business entity, attribute, or relationship has been introduced beyond the supplied ER model. The resulting design is suitable as the basis for subsequent SQL implementation and future semantic-search extensions.

NAME&SRN

1. VASANTH KUMAR REDDY(PES2UG25CS829)

2. MUNIREDDY S (PES2UG25CS813)